https://eap.bl.uk/

1. The International Image Interoperability Framework (IIIF) Manifest

Find a IIIF manifest in EAP

Most collections in the Endangered Archives Programme are published online. This allows you to view the scanned materials on the British Library's site.

It is also possible to access these collections as data. The BL provides access to both metadata and images using the International Image Interoperability Framework (IIIF).

1. Go to https://eap.bl.uk/search
2. Click on "View archives from this project." If you don't see that button, the images are not available yet on the EAP site.
3. Find a File that interests you.
4. Locate the IIIF icon ( ) and get the link. You can click on the icon and select the address. You can also right-click on the icon and select "copy link".
5. You should get something that looks like the manifest URL below.

Manfiest URL

```
manifest_url: " https://eap.bl.uk/archive-file/EAP550-1-3/manifest?manifest "
```

Show code

```
謝雷傷科 [Sutra]
Identifier:EAP550/1/3
Title:謝雷傷科 [Sutra]
Description:A manuscript used in ceremony to invite various spirits.
Extent:65 digital images in TIFF format
Place:China, Asia
Link to catalogue record:<a href="https://eap.bl.uk/archive-file/EAP55
Citation:謝雷傷科 [Sutra], British Library, EAP550/1/3, https://eap.bl.u
Digitised by:Sun Yat-sen University
```

```
Digitisation funded by:Endangered Archives Programme supported by Arca
Language:Chinese
Scripts:Chinese
```

> Some manifests always return a 202 error. We're not sure why. If that happens. Open the manifest URL in your browser. Save it as manifest.json. Then upload the file by clicking the Files icon to the left and then "Upload to session storage." Then change the manifest url above to `manifest.json` and try again.

Beyond serving images and metadata to the Web, the IIIF also allows for many transformations of the image just by changing the URL. You can request a black and white version, a cropped version, or a resized version. See the IIIF Image API documentation for more details.

Download images for processing

```
# needed to run async in Jupyter

import nest_asyncio
nest_asyncio.apply()
```

Sample size

```
# @title Sample size
# At first, we'll only process a few files. However, you can change tl
# For demonstration purposes, we only download the first ten images.
sample_size = 10
```

Download image files

> Often the default image from IIIF is too small to read. So we'll download the full image just like the viewer does on the British Library site (using image tiles)
>
> Show code

Set scale factor

We recommend using the full canvas image. If you get an `Exceeded limit on max bytes per data-uri item` you can download a smaller version of the file. Just reduce the scale factor.

scale_factor: 0.9

Show code

This cell makes a list of tile regions to request using IIIF. It uses the same sample_size as above

Show code

```
  0%|          | 0/10 [00:00<?, ?it/s]fetching  160  tiles
 10%|▉         | 1/10 [00:10<01:33, 10.39s/it]fetching  160  tiles
 20%|█▉        | 2/10 [00:24<01:38, 12.29s/it]fetching  160  tiles
 30%|██▉       | 3/10 [00:36<01:26, 12.37s/it]fetching  160  tiles
 40%|███▉      | 4/10 [00:48<01:14, 12.35s/it]fetching  160  tiles
 50%|████▉     | 5/10 [00:59<00:58, 11.66s/it]fetching  160  tiles
 60%|█████▉    | 6/10 [01:10<00:45, 11.39s/it]fetching  160  tiles
 70%|██████▉   | 7/10 [01:21<00:34, 11.43s/it]fetching  160  tiles
 80%|███████▉  | 8/10 [01:29<00:20, 10.31s/it]fetching  160  tiles
 90%|████████▉ | 9/10 [01:38<00:09,  9.81s/it]fetching  160  tiles
100%|██████████| 10/10 [01:47<00:00, 10.71s/it]
```

Evaluate the images

Before moving forward, it is helpful to manually inspect a few of the images. Could you transcribe the text given the image? As a general rule of thumb, if you're not able to complete the task manually, the model won't be able to either.

Inspect a random page from the downloaded images

Show code

雷司消災保命看經誦宝經祈来吉泰
敕下鑾經童子對讀仙官持經玉女護
經符吏心神安靜思念感通　奉對
帝前開㢟如法按臣各敬志心皈命礼
十方三宝當信心開悟解受持執卷當
平胸座誦念如来未念道經先誦淨身
口神咒　丹珠口神〻　各曉
開經讀誦天尊　此懺文小謝病在尾
爾時信善啟告上聖天尊王曰佛言今此

Connect to Google Drive

We're now going to create a folder in Drive. Feel free to change the name if you like.

The main goal is to create a folder of images that we can access in the next section.

> Start here if you're only working with a folder on Drive and not from a IIIF collection.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Name of the folder to create and save images in Drive

destination_folder_name: " EAP550-1-3 "

Show code

```
import shutil
import os

source_dir = '/content/img'
destination_dir = f'/content/drive/MyDrive/{destination_folder_name}'

os.makedirs(destination_dir, exist_ok=True)

for item in os.listdir(source_dir):
    s = os.path.join(source_dir, item)
    d = os.path.join(destination_dir, item)
    if os.path.isdir(s):
        shutil.copytree(s, d, symlinks=False, ignore=None)
    else:
        shutil.copy2(s, d)
print(f"Contents of {source_dir} copied to {destination_dir}")
```

```
Contents of /content/img copied to /content/drive/MyDrive/EAP550-1-3
```

Using your own images

Very often, researchers work with materials from several archives and collections. It's also very common to take photos with your phone in the archives.

If you have these kinds of images, you can add them to your folder in Drive.

The next two cells will covert any PDF files or .HEIC files (iPhone) into jpegs for processing.

```
%pip install -q pymupdf
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.1/24.1 MB 47.9 MB/s eta
```

```
import pymupdf
from tqdm import tqdm

drive_pdfs = Path(destination_dir).glob('**/*.pdf')
for pdf_file in drive_pdfs:
  doc = pymupdf.open(pdf_file)
  for i, page in tqdm(enumerate(doc)):  # iterate through the pages
      pix = page.get_pixmap(dpi=150)
      img = pix.pil_image()
      img.save(f"{pdf_file.parent}/{pdf_file.stem}_{i}.jpg")
```

```
1it [00:00,  8.58it/s]
```

```
%pip install -q pillow-heif
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.5/5.5 MB 81.3 MB/s eta 0
```

```
from PIL import Image
from pillow_heif import register_heif_opener

# Register the HEIF opener
register_heif_opener()

drive_heic = Path(destination_dir).glob('**/*.HEIC')

for heic_file in tqdm(drive_heic):
  img = Image.open(heic_file)
  img.save(f"{heic_file.parent}/{heic_file.stem}.jpg")
```

```
1it [00:01,  1.82s/it]
```

2. Processing Workflow to Extract Text and Metadata

While the descriptions from the first notebook are nice, they're not research data or archival metadata.

This next section details a workflow that will process your materials into a structured format that can be used for research or library systems. This is an example that can be adapted to your specific needs.

Vision-Language Models

This workshop features a relatively new kind of machine learning model. Traditional optical character recognition (OCR) models look for patterns in an image and map them to machine-readable characters. The process is entirely visual. Recent advances in large language models provide a general understanding of natural language. Vision-language models add a visual component to the language model. As a result, the model's predictions are informed by the visual features of the handwritten text image as well as the model's knowledge of language. This combination leads to much improved accuracy with text extraction from handwritten documents.

We should note that VLMs also "hallucinate." They will guess even when they're not sure. In some cases, they will "improve" on your original document by correcting spelling or word choice. While we cannot use these models uncritically, they very often move us from "could this better?" to "good enough."

> Why not use ChatGPT or Claude?
> While proprietary frontier models are powerful, they can be very expensive to use at scale. Using open models like Qwen allows you to have more control over costs and data privacy. Additionally, we're using a smaller model (8 billion parameters versus 1 trillion) that is focused on a specific task.

```
!pip install -q langchain==0.3.27 langchain-core==0.3.79 langgraph ne
```

```
                                        1.0/1.0 MB 39.3 MB/s eta 0
                                        449.8/449.8 kB 27.2 MB/s e
                                        155.4/155.4 kB 11.9 MB/s e
```

Sending your files to the model

Before we can send your files to the model, we first need to encode them.

The most common way to do this is to encode the binary data in your file as a string of characters using the base64 encoding system. So rather than sending ones and zeros over the Internet, we'll be sending packets of text characters. On the other end, the model turns that data back into an image file to process.

Encode images for transfer over the Internet

Show code

Add the API Key

For this workshop, we'll provide a key that you can use to connect with Alibaba's servers in Singapore. The model that we're using is open-source and can be run locally or on high performance computing.

To add the API key,

- click on the key icon to the left.
- Add new secret.
- Under name enter: DASHSCOPE_API_KEY
- Under Value enter: sk-...

Please note that the key will not work after the workshop. You are welcome to get a key yourself or connect with any of the other providers listed here. To create a key with Alibaba:

- Create an Alibaba Cloud account
- Create an API key

```
from google.colab import userdata

base_url="https://dashscope-intl.aliyuncs.com/compatible-mode/v1"
api_key= userdata.get('DASHSCOPE_API_KEY') # We will provide you with
model = "qwen3-vl-plus" #or "qwen3-vl-flash" (faster and cheaper), "q
```

While we're using Alibaba for the workshop, we are deliberately model and provider agnostic.

Hugging Face 🤗

```
base_url="https://router.huggingface.co/v1"
api_key=userdata.get('HF_TOKEN')
model="Qwen/Qwen3-VL-30B-A3B-Instruct:novita"
```

OpenAI

```
base_url="https://api.openai.com/v1"
api_key=userdata.get('OPENAI_API_KEY')
```

```
model = 'gpt-5-mini'
```

Anthropic

```
base_url="https://api.anthropic.com/v1"
api_key=userdata.get('ANTHROPIC_API_KEY')
model = 'claude-sonnet-4-20250514'
```

Code that sends the encoded image to the server

Show code

State

As we move from step to step in the workflow, we will need something to store the data from prior steps and to make that data available. This object is often called "state" in programming. In the cell below, we create a simple state object to hold our data as we move through the workflow.

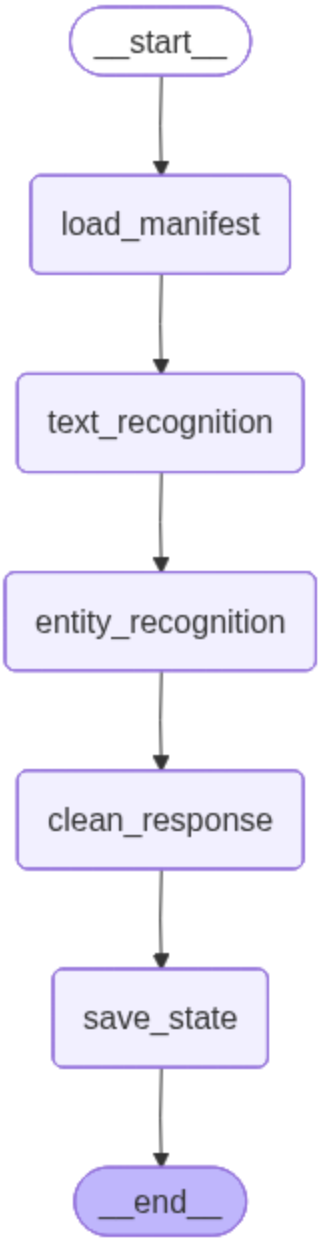
Given that we're processing a box of archival materials, I have called it "Box." I find it helpful to think of the fields in the object as columns in a spreadsheet. Each field (box_manifest_uri, image_uris...) gives us a placeholder for information that we'd like to gather during the workflow. How you design your state object can be helpful in deciding what steps are needed in the workflow and what your final output should look like.

```
# https://langchain-ai.github.io/langgraph/tutorials/workflows/#prompt
# https://typing.python.org/en/latest/spec/typeddict.html

from typing_extensions import TypedDict

class Box(TypedDict):  # Level: File in EAP
    image_path: str
    box_metadata: dict
    file_lookup: dict
    images: list[dict]
```

Steps in the Workflow



Code that details what to do at each step of the workflow

Show code

```
from langgraph.graph import StateGraph, START, END


# Build workflow
builder = StateGraph(Box)

# Add nodes
builder.add_node("load_manifest", load_manifest)
builder.add_node("text_recognition", text_recognition)
```

```
builder.add_node("entity_recognition", entity_recognition)
builder.add_node("clean_response", clean_response)
builder.add_node("save_state", save_state)

# Add edges to connect nodes
builder.add_edge(START, "load_manifest")
builder.add_edge("load_manifest", "text_recognition")
builder.add_edge("text_recognition", "entity_recognition")
builder.add_edge("entity_recognition", "clean_response")
builder.add_edge("clean_response", "save_state")
builder.add_edge("save_state", END)

# Compile
graph = builder.compile()
```

```
# Invoke

output = graph.invoke({"image_path":f"/content/drive/MyDrive/{destina
```

```
🔍 Starting text recognition...
Processing images: 100%|██████████| 65/65 [00:39<00:00,  1.66it/s]
🏷️ Starting entity recognition...
Extracting entities from  images: 100%|██████████| 65/65 [00:39<00:00,
🧹 Cleaning responses...
Cleaning text: 100%|██████████| 65/65 [00:00<00:00, 10016.52it/s]
💾 Saved processed state to output/EAP550_1_3.json
```

```
# inspect results
import random
from IPython.display import Image

images = output.get('images')
image = random.choice(images)

print(image)
Image(url=image["image_uri"], width=400)
```


```
{
    'name': 'EAP550_1_3_1.jpg',
    'markdown_text': '```markdown\n奉道正一謝雷祈受安炁令震脩香灯\n酒食凡供之
净掃蓬莱山上路 氣迎雷帝降凡筵\n五色彩雲随步起 六珠仙服着身来
滿千\n妙哉三洞响\n尚未献凡供略異歆嘗伏望帝慈俯垂\n洞鑒伏以一欵投誠仰瀆雷神之听凡\n恍
得之聞具有疏文謹\n當宣讀\n\n云南省民族古籍整理出版规划办公室 0052\n```',
    'entities': [
        {
            'entity': '谢雷祈',
            'type': 'Person',
            'context': 'A person associated with the ritual or religiou
practitioner or officiant of the ceremony.'
```

```
        },
        {
            'entity': '雷帝',
            'type': 'Deity',
            'context': 'A divine figure invoked in the ritual, represe
in Daoist tradition.'
        },
        {
            'entity': '蓬莱山',
            'type': 'Place',
            'context': 'A mythical mountain in Chinese mythology, ofte
paradise; here it refers to the sacred site where the ritual is taking
        },
        {
            'entity': '云南省民族古籍整理出版规划办公室',
            'type': 'Organization',
            'context': 'An official body responsible for the preservat
minority historical and cultural texts in Yunnan Province, China. The
the text.'
        },
        {
            'entity': '0052',
            'type': 'Identifier',
            'context': 'A catalog or reference number assigned by the云
particular document or entry.'
        }
    ],
    'cleaned_text': '奉道正一謝雷祈受安炁令震脩香灯\n酒食凡供之儀列在聖前羽衆運
氣迎雷帝降凡筵\n五色彩雲随步起 六珠仙服着身来
滿千\n妙哉三洞响\n尚未献凡供略異歆嘗伏望帝慈俯垂\n洞鑒伏以一欵投誠仰瀆雷神之听凡\n
得之聞具有疏文謹\n當宣讀\n\n云南省民族古籍整理出版规划办公室 0052',
    'image_uri': ''
}
```


3. Saving and Publishing

Now that you've run the workflow in the previous notebook, it's time to turn that data into something familiar that you can use. In this section, we'll turn the output JSON file into a spreadsheet and a static website.

Create a spreadsheet

```
import shutil
from pathlib import Path

output_path = '/content/output'
```

```
data_files = Path(output_path).glob("*.json")
data_files = list(data_files)
print(f"Found {len(data_files)} file(s)")
```

```
Found 1 file(s)
```

```
# name, image_uri, cleaned_text, box_metadata>"Idenfifier"
import srsly
from csv import DictWriter


def output_json_to_spreadsheet(data_files: list):
  rows = []
  for data_file in data_files:
    data = srsly.read_json(data_file)
    box_identifier = data.get('box_metadata', 'unlabeled_box').get('I
    images = data.get('images',[])
    # sort the images by filename
    images = sorted(images, key=lambda x: x.get('name',''))
    for image in images:
      rows.append({
          'name': image.get('name',''),
          'image_uri': image.get('image_uri',''),
          'cleaned_text': image.get('cleaned_text',''),
          'box_identifier': box_identifier
      })
  writer = DictWriter(open(data_file.with_suffix('.csv'),'w'), fieldn
  writer.writeheader()
  writer.writerows(rows)

output_json_to_spreadsheet(data_files)
```

Save the output.csv file to Drive

```
# sync content in folders
import shutil

source_dir = '/content/output'
destination_dir = f"/content/drive/MyDrive/{destination_folder_name}/

shutil.copytree(source_dir, destination_dir, dirs_exist_ok=True)
```

```
'/content/drive/MyDrive/EAP550-1-3/output/'
```

With the file saved, you can now go to Drive to work with your data.

- In your folder, you'll find an `output` subfolder
- click to open the `output.csv` file

- Then click on "Open in Google Sheets" (center top of the page)

Create a static website

```
import srsly
import markdown
from pathlib import Path

if not Path('_site').exists():
  Path('_site').mkdir()

for data_file in data_files:
    data = srsly.read_json(data_file)
    images = data.get('images',[])
    images = sorted(images, key=lambda x: x.get('name',''))
    first_image = images[0]['name'] + ".html"
    last_image = images[-1]['name'] + ".html"
    for idx, image in enumerate(images):
        next_image = images[idx+1]['name'] + ".html" if idx < len(ima
        prev_image = images[idx-1]['name'] + ".html" if idx > 0 else
        cleaned_text = image.get('cleaned_text','')
        cleaned_text = markdown.markdown(cleaned_text, extensions=['t
        sort_value = int("".join([char for char in image["name"] if c
        html_template = f"""
        <!DOCTYPE html>
        <html>
          <body>
            <button><a href="/">Back to Search</a></button> &nbs
            <button><a href="{prev_image}">Back</a></button>
            <button><a href="{next_image}">Forward</a></button>

            <h1 data-pagefind-sort="page:{sort_value}">{image["name"]
            <img src="{image["image_uri"]}">
            <p>{cleaned_text}</p>
            <table>
              <tr>
                <th>Entity</th>
                <th>Type</th>
                <th>Context</th>
              </tr>
              {[f"<tr><td>{entity["entity"]}</td><td>{entity["type"]}
            </table>
          </body>
        </html>
        """
        Path(f"_site/{image['name']}.html").write_text(html_template)
```

```
full_metadata_html = ""
for data_file in data_files:
    data = srsly.read_json(data_file)
    box_metadata = data.get('box_metadata',{})
    identifier = box_metadata.get('Identifier','no_identifier')
    full_metadata_html += f"<h1>{identifier}</h1>"
    for key, value in box_metadata.items():
      full_metadata_html += f"<h2>{key}</h2>"
      full_metadata_html += f"<p>{value}</p><br>"
    full_metadata_html += "<hr>"

index = f"""

<!DOCTYPE html>
        <html>
          <head>
          <link href="/pagefind/pagefind-ui.css" rel="stylesheet">
          <script src="/pagefind/pagefind-ui.js"></script>
          </head>
          <body>
            <div id="search"></div>
            {full_metadata_html}"""
index += """
            <script>
                window.addEventListener('DOMContentLoaded', (event) =>
                    new PagefindUI({
                      element: "#search",
                      sort: { date: "desc" },
                      showSubResults: true
                    });

                });
            </script>
             </body>
        </html>
"""
Path('_site/index.html').write_text(index)
```

```
1716
```

Create a search index

```
%pip install -q 'pagefind[extended]'
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 57.6/57.6 MB 16.9 MB/s eta
```

```
!python3 -m pagefind --site _site
```

```
Running Pagefind v1.4.0 (Extended)
Running from: "/content"
Source:       "_site"
Output:       "_site/pagefind"

[Walking source directory]
Found 3 files matching **/*.{html}

[Parsing files]
Did not find a data-pagefind-body element on the site.
↳ Indexing all <body> elements on the site.

[Reading languages]
Discovered 1 language: unknown

[Building search indexes]
Total:
  Indexed 1 language
  Indexed 3 pages
  Indexed 541 words
  Indexed 0 filters
  Indexed 1 sort

Finished in 0.031 seconds
```

```
import shutil
zip_file = shutil.make_archive('site', 'zip', '/content/_site')
```

```
# Save the zip file to Drive
import shutil

source_file = '/content/site.zip'
destination_dir = f"/content/drive/MyDrive/{destination_folder_name}/(

shutil.copy(source_file, destination_dir)
```

```
'/content/drive/MyDrive/EAP285-11-8/output/site.zip'
```

```
from google.colab import files
files.download(zip_file)
```

Open the website on your computer

You can then unzip the `site.zip` file on your computer and view it in your web browser.

- Open your terminal application

- Open your terminal application
- Navigate to the unzipped folder (ex. `cd ~/Downloads/site`)
- Enter this command `python -m http.server` and press enter.
- The site should now be visible in the browser at http://localhost:8000/

Deploy your website to the web

There are many ways to put your site up on the web. One of the simplest is a company called Netlify, which has a nice free tier.

- Create an account with Netlify: https://app.netlify.com/signup
- Login in to the new account
- Select the Projects tab on the left
- Then select Add New Project
- In the dropdown, select Deploy Manually
- Drag and drop the unzipped `site` folder from your computer
- Once uploaded, you should see a URL for your site. Something like `33242-fish-pickle.netlify.app`
- You can change the web address in General > Project Details > Change Project Name
- For example, I changed the address to: https://eap617-1-10.netlify.app

Collaborations with Libraries, Archives and Museums

While it's great to have a spreadsheet or static page to work from, ideally we'll have ways for transcriptions and data to find their way back to the British Library and other libraries and museums. Librarians and archivists are hard at work developing this capability. For example, the Swedish National Archives, recently added the ability to search over 3 million hand-written documents.

- Here is a post about this work (in Swedish and English)
- Here is an example search result

Disclosure of Delegation to Generative AI

The authors declare the use of generative AI in the research and writing process. According to the GAIDeT taxonomy (2025), the following tasks were delegated to GAI tools under full human supervision:

- Code generation
- Code optimization

The GAI tool used was: Claude Sonnet 4.5.
Responsibility for the final manuscript lies entirely with the authors. GAI tools are not listed as authors and do not bear responsibility for the final outcomes. Declaration submitted by: Andy Janco